
type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin IF3270]]

[!cornell] Motivasi dan Konsep Dasar CNN

Questions/Cues

- Mengapa ANN tidak efektif untuk klasifikasi gambar?
- Bagaimana CNN mengatasi masalah dimensi tinggi?
- Apa keunggulan model dalam dibanding dangkal?
- Mengapa CNN lebih efisien secara komputasi?
- Bagaimana cara kerja visi komputer vs manusia?

Reference Points

- Lecture_CNN.pptx (Halaman 6-19, 34-37)
- Deep Learning Book Reference (Halaman 8-10)
- Khan Survey Paper (Halaman 12)

Keterbatasan ANN untuk Pemrosesan Gambar

Jaringan saraf tiruan tradisional (ANN) menghadapi tantangan signifikan saat memproses data gambar. Gambar digital direpresentasikan sebagai array multidimensi piksel ($\text{tinggi} \times \text{lebar} \times \text{saluran warna}$). Untuk gambar beresolusi 1024×1024 RGB, ANN harus memproses lebih dari 3 juta parameter input. Pendekatan flattening (meratakan gambar menjadi vektor 1D) menyebabkan hilangnya fitur spasial penting seperti tepi, pola lokal, dan hubungan geometris antar objek. Contoh konkret: Gambar kucing 32×32 piksel RGB akan menjadi vektor input dengan 3.072 fitur ($32 \times 32 \times 3$). Lapisan tersembunyi pertama dengan 1.000 neuron akan menghasilkan lebih dari 3 juta parameter yang harus dilatih. Ini menyebabkan masalah komputasi dan berpotensi overfitting ketika dataset terbatas. Masalah ini dikenal sebagai kutukan dimensionalitas (curse of dimensionality), di mana kompleksitas model tumbuh secara eksponensial dengan dimensi input.

Peran Deep Learning dan CNN

Convolutional Neural Network (CNN) muncul sebagai solusi biologis yang terinspirasi dari sistem visual mamalia. Berbeda dengan ANN tradisional, CNN mempertahankan struktur spasial gambar melalui operasi konvolusi hierarkis. Lapisan awal mendeteksi fitur dasar seperti tepi dan tekstur, sementara lapisan yang lebih dalam mengkombinasikannya menjadi bentuk kompleks seperti mata atau bulu. Keunggulan utama CNN terletak pada kemampuan ekstraksi fitur otomatis. Contoh praktis: Tanpa pemrograman eksplisit, CNN dapat mempelajari bahwa keberadaan kombinasi tepi melengkung dan garis vertikal mungkin mengindikasikan keberadaan roda mobil. Efisiensi komputasi CNN berasal dari penggunaan kernel yang dibagi parameter (parameter sharing) dan konektivitas lokal, walau mekanisme ini telah dibahas di materi lain.

Keuntungan Arsitektur Dalam

Teorema Universal Approximator menyatakan bahwa jaringan saraf dengan satu lapisan tersembunyi dapat mengaproksimasi fungsi apa pun. Namun praktik menunjukkan bahwa model dalam (deep) memiliki keunggulan generalisasi lebih baik. Pada eksperimen LeCun (1989), CNN dengan 4 lapisan mencapai akurasi 98.4% pada pengenalan digit tulisan tangan, sementara ANN dangkal hanya 87% dengan parameter 3x lebih banyak. Alasan utama kedalaman meningkatkan performa: (1) Representasi hierarkis fitur yang analog dengan cara korteks visual mamalia memproses informasi, (2) Penggunaan kembali parameter melalui operasi konvolusi mengurangi risiko overfitting, dan (3) Kemampuan menangkap abstraksi bertingkat dari piksel → tepi → pola → objek parsial → objek utuh.

Efisiensi Komputasional CNN

CNN mencapai efisiensi melalui dua prinsip utama (tanpa menjelaskan mekanisme teknis). Untuk gambar 320×280 , operasi deteksi tepi dengan kernel 3×3 hanya memerlukan ~268.000 operasi, sangkan ANN memerlukan >8 miliar operasi. Pengurangan kompleksitas ini memungkinkan pelatihan model pada hardware konsumen modern. Contoh historis: Arsitektur LeNet-5 (1998) untuk pengenalan digit tulisan tangan hanya membutuhkan 60.000 parameter dibandingkan jutaan pada ANN setara. Model ini memanfaatkan pola lokal dalam gambar melalui operasi konvolusi berlapis, yang kemudian menjadi fondasi arsitektur CNN modern seperti AlexNet dan ResNet.

[!cornell] ##### Summary

Convolutional Neural Network (CNN) merupakan terobosan penting dalam deep learning untuk pemrosesan data grid-like seperti gambar. **Keunggulan utama CNN**

terletak pada preservasi fitur spasial dan efisiensi komputasi melalui operasi konvolusi berlapis. Dibanding ANN tradisional, CNN **mengurangi parameter secara signifikan** sambil **meningkatkan akurasi** pada tugas visi komputer. Arsitektur dalam CNN memungkinkan **ekstraksi fitur hierarkis otomatis** dari level rendah (tepi, tekstur) hingga tinggi (bentuk kompleks).

[!ad-libitum]- Additional Information

Analisis Matematis Kutukan Dimensionalitas

Masalah dimensionalitas formal dapat dimodelkan dengan ruang fitur berdimensi tinggi. Untuk dataset dengan d fitur independen dan n sampel, kepadatan data berkurang secara eksponensial sebagai $(1/n)^{(1/d)}$. Pada gambar 100×100 RGB ($d=30.000$), dibutuhkan dataset berukuran $10^{30.000}$ untuk mencapai kepadatan setara dataset 1D dengan 10 sampel. CNN mengatasi ini melalui translasi invarian dan ekstraksi fitur lokal.

Evolusi Historis CNN

Perkembangan CNN melalui lima era: (1) Prinsip dasar oleh Fukushima (1980), (2) Implementasi praktis LeNet (1990), (3) Kebangkitan kembali pasca ImageNet 2012, (4) Dominasi arsitektur seperti VGG/ResNet, (5) Inovasi kontemporer dengan attention mechanisms. Kemajuan hardware (GPU) dan dataset besar (ImageNet) menjadi katalis utama adopsi CNN.

Perbandingan dengan Model Deep Learning Lain

Perbandingan Arsitektur - **RNN**: Optimal untuk data sekuensial (teks/sinyal temporal) - **Autoencoder**: Fokus pada kompresi dan rekonstruksi fitur - **Transformers**: Dominan di NLP dengan mekanisme attention - **CNN**: Tetap unggul untuk tugas visi murni karena bias induktif spasial

Proyek Eksplorasi Mandiri

1. Implementasikan MLP sederhana untuk klasifikasi MNIST dan bandingkan kompleksitas parameternya dengan CNN sederhana
2. Eksperimen dengan dimensionality reduction (PCA) pada fitur gambar sebelum input ke ANN, analisis dampak pada akurasi
3. Visualisasi perbedaan representasi fitur di lapisan awal vs akhir CNN menggunakan teknik seperti DeepDream

Bacaan Lanjutan

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning (Bab 9: Convolutional Networks)
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). [Deep learning review in Nature](#)
- Khan, A., et al. (2020). [Survey of CNN Architectures](#)
- Kursus Online: [Deep Learning Specialization \(Coursera\)](#)

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin]]

[!cornell] Local Connectivity and Parameter Sharing Mechanisms

Questions/Cues

- Mengapa konektivitas lokal meningkatkan generalisasi CNN?
- Bagaimana mekanisme berbagi parameter mengurangi kompleksitas?
- Apa perbedaan receptive field pada CNN vs ANN tradisional?
- Bagaimana eksperimen LeCun (1989) mendemonstrasikan manfaat parameter sharing?
- Mengapa operasi konvolusi lebih efisien secara komputasi?

Reference Points

- IF3270 Pembelajaran Mesin Slides (Halaman 20-34)
- Goodfellow et al., Deep Learning (Halaman 22, 28, 34)
- LeCun (1989) Generalization and Network Design Strategies

Konsep Konektivitas Lokal

Konektivitas lokal merupakan prinsip dasar dalam CNN di mana setiap neuron pada lapisan konvolusional hanya terhubung ke subset kecil neuron pada lapisan sebelumnya. Ini meniru konsep **receptive field** pada sistem visual biologis yang ditemukan Hubel dan Wiesel (1959), di mana sel-sel saraf tertentu hanya merespons stimulus dari area terbatas pada retina. Pada CNN, receptive field didefinisikan sebagai area input yang mempengaruhi nilai satu unit pada feature map. Contoh konkret: Untuk kernel berukuran 3x3, setiap neuron di feature map

hanya terhubung ke 9 piksel input (pada satu channel), bukan ke seluruh gambar. Ini berbeda dengan ANN tradisional di mana setiap neuron output terhubung ke SEMUA neuron input (koneksi penuh). Pada gambar 32×32 RGB, ANN akan membutuhkan $32 \times 32 \times 3 = 3072$ koneksi per neuron output, sedangkan CNN dengan kernel 3×3 hanya membutuhkan $3 \times 3 \times 3 = 27$ koneksi.

Mekanisme Parameter Sharing

Parameter sharing adalah teknik dimana **filter/kernel yang sama** digunakan di seluruh lokasi spasial pada input. Artinya, sekumpulan bobot yang sama (kernel) diaplikasikan ke setiap bagian gambar melalui operasi konvolusi. Ini berbeda dengan lapisan locally-connected tanpa sharing di mana setiap lokasi spasial memiliki filter unik. Contoh perhitungan parameter: Untuk feature map 9×9 dengan kernel 3×3 : - **Tanpa sharing**: $81 \text{ neuron} \times (9 \text{ bobot} + 1 \text{ bias}) = 810 \text{ parameter}$ - **Dengan sharing**: $1 \text{ kernel} \times (9 \text{ bobot} + 1 \text{ bias}) = 10 \text{ parameter}$ Efisiensi ini memungkinkan CNN mempelajari fitur hierarkis seperti tepi $\rightarrow$ tekstur $\rightarrow$ objek parsial $\rightarrow$ objek utuh dengan parameter jauh lebih sedikit. Eksperimen LeCun (1989) menunjukkan peningkatan akurasi dari 88.5% (locally-connected) ke 98.4% dengan tambahan parameter sharing, sambil mengurangi parameter dari 1226 ke 1060.

Efisiensi Komputasi

Konektivitas lokal dan parameter sharing bersama-sama mengurangi kompleksitas komputasi secara eksponensial. Operasi konvolusi untuk edge detection pada gambar 320×280 : - **ANN**: Membutuhkan ~ 8 miliar operasi ($320 \times 280 \times 319 \times 280$ perkalian matriks) - **CNN**: Hanya $\sim 268,000$ operasi ($319 \times 280 \times 3$ operasi per kernel) Rasio efisiensi mencapai 60,000:1. Pada arsitektur LeNet-5 asli, parameter sharing mengurangi bobot dari 64,660 menjadi hanya 8,824 tanpa penurunan akurasi signifikan. Efisiensi ini memungkinkan pelatihan model deep learning pada hardware terbatas.

Dampak Terhadap Generalisasi

Pembatasan konektivitas dan berbagi parameter bertindak sebagai **regularisasi implisit** dengan: 1. Membatasi kapasitas model untuk overfit 2. Memaksa model belajar fitur invarian terhadap translasi 3. Menginduksi bias pembelajaran hierarki fitur spasial Eksperimen pada Net-3 LeCun menunjukkan akurasi 98.4% dengan parameter sharing vs 88.5% tanpa sharing, membuktikan bahwa pembatasan parameter justru meningkatkan kemampuan generalisasi model pada data unseen.

[!cornell] ##### Summary **Konektivitas lokal** membatasi setiap neuron hanya merespons area terbatas (receptive field), meniru sistem visual biologis dan

mengurangi koneksi redundant. **Parameter sharing** menggunakan filter yang sama di seluruh input, menurunkan kompleksitas model secara eksponensial sambil memaksa pembelajaran fitur invarian spasial. Kombinasi keduanya menghasilkan efisiensi komputasi hingga $60,000\times$ lebih baik dibanding ANN tradisional dan meningkatkan generalisasi model melalui regularisasi implisit, seperti dibuktikan LeCun (1989) dengan peningkatan akurasi 10% pada tugas pengenalan digit.

[!ad-libitum]- Additional Information

Analisis Eksperimen LeCun 1989

Arsitektur Net-3 menggunakan tiga lapisan tersembunyi dengan strategi hybrid: - Lapisan 1: 256 unit, locally-connected 8x8 dengan kernel 3x3 (640 parameter) - Lapisan 2: 64 unit, kernel 5x5 dengan sharing (416 parameter) - Lapisan 3: 16 unit fully-connected (170 parameter) Total parameter dengan sharing: 1,226 vs 206 jika full sharing. Implementasi modern menunjukkan bahwa partial sharing (campuran lokal dan global) sering memberikan trade-off optimal antara fleksibilitas dan efisiensi.

Matematika Efisiensi Parameter

Rumus umum kompleksitas parameter: - **Tanpa sharing:** $N \times ((F \times F \times D) + 1)$ $N = \text{outputHeight} \times \text{outputWidth} \times \text{featureMaps}$ - **Dengan sharing:** $K \times ((F \times F \times D) + 1)$ $K = \text{jumlah feature maps}$

Studi kasus LeNet-5: - Lapisan C3: 16 feature maps 10x10 $\rightarrow 16 \times (5 \times 5 \times 6 + 1) = 2416$ parameter (dengan sharing) - Tanpa sharing: $10 \times 10 \times 16 \times (25 + 1) = 41,600$ parameter $\rightarrow$ Reduksi $17.2\times$

Implementasi Modern

Framework seperti TensorFlow/Keras mengimplementasikan parameter sharing melalui: `python Conv2D(filters=16, kernel_size=(3,3), use_bias=True) # Dengan sharing LocallyConnected2D(filters=16, kernel_size=(3,3)) # Tanpa sharing` Benchmark pada dataset CIFAR-10 menunjukkan Conv2D $3\times$ lebih cepat dalam pelatihan dan menggunakan memori $15\times$ lebih sedikit dibanding LocallyConnected2D dengan akurasi setara.

Proyek Eksplorasi Mandiri

1. **Eksperimen Parameter Sharing:**
2. Bangun dua model CNN pada PyTorch/Keras: satu dengan Conv2D standar, satu dengan lapisan lokal tanpa sharing

3. Bandingkan akurasi, waktu pelatihan, dan penggunaan memori pada dataset MNIST

4. Visualisasi Receptive Field:

5. Gunakan library tf-keras-vis untuk memvisualisasikan area input yang paling mempengaruhi neuron tertentu

6. Bandingkan pola receptive field pada lapisan awal vs lapisan dalam

Bacaan Lanjutan

- [Original Paper] LeCun, Y. (1989). Generalization and network design strategies
- [Book Chapter] Goodfellow et al., Deep Learning Chapter 9: Convolutional Networks
- [Review] Krizhevsky et al. (2017). ImageNet Classification with Deep CNNs
- [Tools] TensorFlow Receptive Field Calculator: <https://distill.pub/2019/computing-receptive-fields/>

type: Note cssclasses: - cornell-notes

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Convolutional Layer Operations and Hyperparameters

Questions/Cues

- Operasi dasar apa yang dilakukan lapisan konvolusi?
- Bagaimana stride mempengaruhi dimensi output?
- Mengapa padding diperlukan dalam konvolusi?
- Peran fungsi aktivasi setelah operasi konvolusi?
- Bagaimana pooling mengurangi dimensi spasial?
- Rumus menghitung dimensi output konvolusi?

Reference Points

- IF3270 Convolutional Neural Network (Slides 35-49)
- Goodfellow et al., Deep Learning (Slides 37-38,44-45)

Operasi Konvolusi Dasar

Lapisan konvolusi melakukan operasi matematis berupa **perkalian elemen-bijaksana (element-wise multiplication)** antara kernel/filter dengan bagian input yang sesuai, diikuti oleh **penjumlahan hasil perkalian**. Kernel bergerak melintasi seluruh input dengan pola sistematis untuk mengekstraksi fitur spasial. Contoh konkret: Untuk gambar berwarna 5x5 piksel dengan 3 channel RGB, kernel 3x3x3 akan mengambil sub-region 3x3 dari setiap channel, mengalikannya dengan bobot kernel yang sesuai, lalu menjumlahkan semua nilai untuk menghasilkan satu nilai output pada feature map. Proses ini diulang untuk setiap posisi kernel di seluruh gambar.

Hyperparameter: Stride dan Padding

Stride menentukan jumlah pergeseran kernel saat bergerak melintasi input. Stride 1 berarti kernel bergeser 1 piksel setiap langkah, menghasilkan output yang relatif besar. Stride 2 mengurangi ukuran output menjadi sekitar setengahnya. Contoh praktis: Input 7x7 dengan kernel 3x3 dan stride 2 menghasilkan output 3x3. **Padding** menambahkan lapisan nol di sekitar input untuk mengontrol dimensi output. Padding "valid" berarti tidak ada penambahan, sementara "same" menambahkan padding untuk mempertahankan ukuran input. Contoh aplikasi: Untuk input 5x5 dengan kernel 3x3, padding 1 di semua sisi memungkinkan output tetap 5x5 (dengan stride 1).

Fungsi Aktivasi pada Detector Stage

Setelah operasi konvolusi linier, fungsi aktivasi non-linier seperti **ReLU (Rectified Linear Unit)** diterapkan untuk memperkenalkan non-linearitas. ReLU bekerja dengan mengubah semua nilai negatif menjadi nol ($f(x) = \max(0, x)$). Keuntungan utamanya adalah mencegah masalah vanishing gradient dan mempercepat pelatihan dibanding fungsi sigmoid/tanh. Contoh visual: Jika feature map setelah konvolusi bernilai [-0.5, 2.1, -3.0, 4.7], setelah ReLU menjadi [0, 2.1, 0, 4.7]. Transformasi ini membantu jaringan mempelajari hubungan yang lebih kompleks sambil menjaga komputasi tetap efisien.

Operasi Pooling

Pooling bertujuan untuk **mengurangi dimensi spasial** dan meningkatkan invariansi terhadap pergeseran kecil. **Max-pooling** adalah teknik paling umum yang mengambil nilai maksimum dalam region tertentu (biasanya 2x2). Contoh nyata: Region dengan nilai [5, 8, 3, 6] akan menghasilkan output 8 setelah max-pooling. Average pooling mengambil nilai rata-rata region, sementara L2-norm pooling menggunakan norma L2. Ukuran pooling yang umum adalah 2x2 dengan

stride 2, mengurangi dimensi menjadi setengahnya di setiap sumbu. Operasi ini membantu mengurangi overfitting dan beban komputasi.

Perhitungan Dimensi Output

Dimensi output konvolusi dihitung dengan rumus: $W_{out} = [(W_{in} - F + 2P)/S] + 1$ dimana: - W_{in} : Lebar/tinggi input - F : Ukuran kernel (filter) - P : Jumlah padding - S : Nilai stride Contoh numerik: Input 32x32 dengan kernel 5x5, padding 0, dan stride 1 menghasilkan output 28x28 karena $(32-5+0)/1 + 1 = 28$. Untuk input yang sama dengan stride 2, output menjadi 14x14 $[(32-5+0)/2 + 1 = 14]$.

[!cornell] ##### Summary **Lapisan konvolusi** melakukan ekstraksi fitur melalui operasi sliding kernel dengan mengaplikasikan hyperparameter **stride** untuk mengontrol pergeseran dan **padding** untuk mempertahankan dimensi. Fungsi aktivasi **ReLU** memperkenalkan non-linearitas penting setelah konvolusi, sementara **pooling** bertugas mengurangi dimensi spasial dan meningkatkan invariansi. Perhitungan dimensi output bergantung pada kombinasi parameter input, kernel, stride, dan padding melalui formula matematis yang terdefinisi dengan jelas.

[!ad-libitum]- Additional Information

Analisis Kompleksitas Komputasi

Kompleksitas operasi konvolusi bergantung pada ukuran input ($W \times H \times C_{in}$), ukuran kernel ($K \times K$), dan jumlah filter (C_{out}). Operasi dasar yang diperlukan adalah $O(W \times H \times C_{in} \times K^2 \times C_{out})$. Teknik optimasi seperti separable convolutions dapat mengurangi kompleksitas menjadi $O(W \times H \times C_{in} \times K \times C_{out})$ dengan memisahkan konvolusi spasial dan channel mixing.

Contoh implementasi efisien menggunakan im2col (image to column) yang mengubah operasi konvolusi menjadi perkalian matriks, memanfaatkan BLAS (Basic Linear Algebra Subprograms) yang sangat dioptimalkan pada hardware modern.

Kasus Khusus: Dilasi dan Konvolusi Terpisah

Dilated convolution memperkenalkan celah (gap) dalam kernel untuk memperluas receptive field tanpa meningkatkan parameter. Misalnya, kernel 3x3 dengan dilatasi 2 memiliki receptive field efektif 5x5. Teknik ini berguna dalam segmentasi citra medis dimana konteks spasial luas diperlukan.

Depthwise separable convolution memisahkan operasi menjadi depthwise convolution (beroperasi pada setiap channel terpisah) diikuti pointwise convolution (kombinasi cross-channel). Metode ini mengurangi parameter secara signifikan, digunakan dalam MobileNet untuk aplikasi edge computing.

Project Eksplorasi Mandiri

1. Implementasikan operasi konvolusi manual pada gambar grayscale menggunakan NumPy tanpa library deep learning. Bandingkan hasil dengan fungsi konvolusi OpenCV.
2. Eksperimen dengan kombinasi stride dan padding berbeda pada model CNN sederhana untuk dataset CIFAR-10. Amati pengaruhnya terhadap akurasi dan waktu pelatihan.

Alat Implementasi Praktis

- **TensorFlow Playground:** <https://playground.tensorflow.org> untuk visualisasi interaktif
- **CNN Explainer:** <https://poloclub.github.io/cnn-explainer/> untuk animasi operasi layer
- **PyTorch Conv2D Documentation:** Tutorial resmi implementasi konvolusi <https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Bacaan Lanjutan

- "A guide to convolution arithmetic for deep learning" (Vincent Dumoulin, Francesco Visin)
- "Understanding Convolution in Deep Learning" (Tim Dettmers)
- Kursus Deep Learning Specialization - Coursera (Andrew Ng) Modul 4: Convolutional Networks

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin]]

[!cornell] CNN Architectural Design Patterns

Questions/Cues

- Mengapa LeNet-5 menggunakan urutan konvolusi-pooling-berulang?
- Bagaimana evolusi arsitektur CNN meningkatkan akurasi model?
- Apa fungsi lapisan FC dalam arsitektur CNN modern?
- Bagaimana perbedaan pola desain LeNet-5 vs CNN-XGBoost?

- Mengapa arsitektur CNN cenderung semakin dalam?

Reference Points

- IF3270_Pembelajaran_Mesin.pptx (Halaman 12, 37-40, 52-58)
- Deep Learning (Goodfellow et al.) (Halaman 326-330)
- LeCun (1998) Gradient-based learning applied to document recognition

Sejarah Perkembangan Arsitektur CNN

Arsitektur Convolutional Neural Network (CNN) telah mengalami lima era perkembangan utama sejak kemunculannya pada 1980-an. Awalnya terinspirasi dari sistem visual biologis, CNN pertama seperti LeNet-5 dirancang untuk tugas sederhana seperti pengenalan digit tulisan tangan. Pada era stagnasi (1990-2000), keterbatasan komputasi menghambat pengembangan model yang lebih dalam. Kebangkitan terjadi pada 2012 dengan keberhasilan AlexNet dalam ImageNet, diikuti ledakan inovasi arsitektur seperti VGG, ResNet, dan Transformer-based models. Perkembangan ini ditandai dengan peningkatan kedalaman jaringan (dari 5 lapisan di LeNet-5 hingga >100 lapisan di ResNet), teknik koneksi residual, serta integrasi mekanisme perhatian (attention). Pola desain modern sering mengadopsi blok modular yang dapat diulang, berbeda dengan pendekatan sekuensial linier di arsitektur awal.

Pola Desain Dasar CNN

Arsitektur CNN umumnya mengikuti pola berulang: **Konvolusi** → **Aktivasi** → **Pooling** yang diulang beberapa kali sebelum lapisan Fully Connected (FC). Contoh pada LeNet-5: 1. **Input** → **Conv1** (6 filter 5x5) → **AvgPool1** (2x2) 2. **Conv2** (16 filter 5x5) → **AvgPool2** (2x2) 3. **Flatten** → **FC1** (120 neuron) → **FC2** (84 neuron) → **Output** Pola ini memungkinkan ekstraksi fitur hierarkis: lapisan awal menangkap pola lokal (tepi, tekstur), lapisan tengah mengombinasikannya menjadi bagian objek (mata, telinga), lapisan akhir mengenali objek utuh. Implementasi modern menggantikan average pooling dengan max pooling dan menambahkan normalisasi batch setelah konvolusi.

Variasi Arsitektur Modern

Hybrid CNN-XGBoost: Mengganti lapisan FC dengan Extreme Gradient Boosting untuk klasifikasi multi-label. Contoh implementasi:

```
python base_model = VGG16(weights='imagenet', include_top=False) x = base_model.output x = GlobalAveragePooling2D()(x) predictions = XGBClassifier().fit(x_train, y_train).predict(x) model = Model(inputs=base_model.input,
```

outputs=predictions) **Arsitektur Multi-Input:** Memproses teks dan gambar secara paralel: 1. **Branch Gambar:** Seri konvolusi-pooling 2. **Branch Teks:** Embedding → Conv1D → Pooling 3. **Gabungan:** Concatenate → FC → Output **Desain Modular** (contoh: Inception Module): Menggunakan beberapa ukuran filter secara paralel (1x1, 3x3, 5x5) lalu menggabungkan hasilnya, memungkinkan jaringan memilih kombinasi filter optimal secara otomatis.

[!cornell] ##### Summary

Pola desain arsitektur CNN berevolusi dari struktur sekuensial sederhana (LeNet-5) menjadi jaringan modular dalam-dalam dengan koneksi residual dan mekanisme perhatian. **LeNet-5 memperkenalkan pola dasar** konvolusi-pooling-berulang yang masih menjadi fondasi desain modern, sementara arsitektur hybrid seperti **CNN-XGBoost menunjukkan fleksibilitas** integrasi dengan metode lain. Tren utama meliputi peningkatan kedalaman jaringan, penggunaan blok bangun berulang, dan optimasi untuk komputasi efisien. **Implementasi praktis** memanfaatkan framework seperti Keras/TensorFlow dengan pola layer yang terstandarisasi.

[!ad-libitum]- Additional Information

Analisis Komparatif Arsitektur

Arsitektur	Kedalaman	Inovasi Kunci	Akurasi Top-1 ImageNet
LeNet-5 (1998)	7 layer	Pooling bertingkat	99.2% (MNIST)
AlexNet (2012)	8 layer	ReLU, Dropout	63.3%
VGG-16 (2014)	16 layer	3x3 filter bertumpuk	71.5%
ResNet-50 (2015)	50 layer	Skip connections	76.6%

Pola Emergen: Arsitektur terkini mengadopsi "network-in-network" melalui 1x1 convolution untuk reduksi dimensi dan inverted residual blocks pada model mobile-optimized seperti MobileNetV3.

Implementasi dengan Framework Modern

Contoh implementasi LeNet-5 menggunakan TensorFlow 2.x: `python model = tf.keras.Sequential([ layers.Conv2D(6, (5,5), activation='tanh', input_shape=(32,32,1)), layers.AveragePooling2D(), layers.Conv2D(16, (5,5), activation='tanh'), layers.AveragePooling2D(), layers.Flatten(), layers.Dense(120, activation='tanh'), layers.Dense(84, activation='tanh'),`

`layers.Dense(10, activation='softmax') ])` Modifikasi kinerja tinggi: Ganti aktivasi tanh → ReLU, tambahkan BatchNormalization setelah setiap konvolusi.

Proyek Eksplorasi Mandiri

1. **Implementasi Arsitektur Historis:** Bangun LeNet-5 dari nol menggunakan NumPy saja, bandingkan performa dengan versi TensorFlow pada dataset MNIST.
2. **Desain Arsitektur Hibrid:** Kombinasikan CNN dengan Transformer untuk klasifikasi video, menggunakan CNN untuk ekstraksi fitur spasial dan Transformer untuk pemodelan temporal.
3. **Optimasi untuk Edge Devices:** Modifikasi VGG16 dengan teknik depthwise separable convolution (MobileNet) dan ukur trade-off akurasi vs latency pada Raspberry Pi.

Alat dan Sumber Daya

- **Visualisasi Arsitektur:** Netron (<https://netron.app/>) untuk melihat diagram model
- **Repository Model:** TensorFlow Hub (<https://tfhub.dev/>) untuk arsitektur pra-latih
- **Dataset Benchmark:** ImageNet-1K, CIFAR-100, Pascal VOC
- **Buku Lanjutan:** "Deep Learning for Computer Vision" (Rajalingappaa Shanmugamani)

Bacaan Lebih Lanjut

- [Paper] "Deep Residual Learning for Image Recognition" (He et al., 2015)
- [Tutorial] "CNN Architectures: From LeNet to EfficientNet" (Paperspace Blog)
- [Buku] "Hands-On Neural Networks with Keras" (Leonardo De Marchi)
- [Kursus] "Advanced Computer Vision with TensorFlow" (Coursera Specialization)

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin Lanjut]]

[!cornell] Backpropagation dalam Jaringan Konvolusional

Questions/Cues

- Mengapa backpropagation diperlukan dalam CNN?

- Bagaimana gradien dihitung untuk kernel konvolusi?
- Apa perbedaan backpropagation di CNN vs ANN?
- Bagaimana implementasi praktis backpropagation CNN?
- Tantangan optimasi parameter CNN?

Reference Points

- IF3270 Pembelajaran Mesin (Slides 62-73)
- Goodfellow et al., Deep Learning (Ch. 6)

Tinjauan Umum Backpropagation dalam CNN

Backpropagation pada Convolutional Neural Networks (CNN) merupakan proses kalkulasi gradien error terhadap parameter jaringan (termasuk kernel konvolusi) untuk memperbarui bobot melalui algoritma optimasi seperti Stochastic Gradient Descent. Proses ini memungkinkan CNN mempelajari fitur hierarkis secara otomatis dari data pelatihan. Pada CNN, terdapat dua jenis parameter yang perlu diperbarui: bobot pada lapisan fully-connected dan nilai kernel pada lapisan konvolusi. Kompleksitas utama terletak pada perhitungan gradien untuk kernel konvolusi karena sifat operasi konvolusi yang melibatkan penerapan kernel yang sama pada seluruh wilayah input (parameter sharing). Contoh: Pada jaringan dengan satu lapisan konvolusi diikuti lapisan fully-connected, gradien error harus dipropagasi balik melalui operasi matriks fully-connected terlebih dahulu, kemudian melalui operasi konvolusi untuk memperbarui nilai kernel.

Perhitungan Gradien pada Lapisan Konvolusional

Untuk menghitung gradien kernel konvolusi (K), digunakan aturan rantai (chain rule) melalui empat komponen utama: 1. **Gradien error terhadap output** ($\partial E / \partial O$): Bergantung pada fungsi loss yang digunakan 2. **Gradien output terhadap net** ($\partial O / \partial \text{net}$): Bergantung pada fungsi aktivasi 3. **Gradien net terhadap feature map** ($\partial \text{net} / \partial H$): Matriks bobot lapisan berikutnya 4. **Gradien feature map terhadap kernel** ($\partial H / \partial K$): Input asli yang dikonvolusi Rumus umum gradien kernel: $\partial E / \partial K = \partial E / \partial O * \partial O / \partial \text{net} * \partial \text{net} / \partial H * \partial H / \partial K$ Dimana $\partial H / \partial K$ dihitung melalui operasi konvolusi antara input (X) dan gradien dari lapisan berikutnya. Analogi: Bayangkan mengoreksi kesalahan dalam mendeteksi tepi gambar. Kesalahan di lapisan output dipetakan balik ke kernel konvolusi dengan melihat area mana pada gambar input yang paling berkontribusi terhadap kesalahan tersebut.

Contoh Implementasi Langkah Demi Langkah

Kasus: CNN sederhana dengan arsitektur Input $\rightarrow$ Konvolusi (1 kernel 3×3) $\rightarrow$ Fully Connected $\rightarrow$ Output

Langkah 1: Forward Pass - Input X = matriks 5×5 - Konvolusi dengan kernel K (3×3) menghasilkan feature map H (3×3) - H di-flatten menjadi vektor - Fully connected layer dengan bobot W menghasilkan output O

Langkah 2: Backward Pass

1. Hitung $\partial E / \partial W$ menggunakan error di output: $\partial E / \partial W = (O - \text{target}) * \text{turunan aktivasi} * H$
2. Hitung gradien untuk kernel K : $\partial E / \partial K = X \otimes (\partial E / \partial H)$ Dimana $\otimes$ menyatakan operasi konvolusi valid dan $\partial E / \partial H = (\partial E / \partial O * \partial O / \partial \text{net} * W)$

Contoh numerik: Jika error di output adalah 2.5 dan turunan ReLU di feature map adalah 0.8, gradien kernel dihitung dengan mengkonvolusi input asli dengan gradien yang telah dipropagasi.

Tantangan dan Pertimbangan Praktis

1. **Vanishing Gradients:** Lapisan konvolusi bertingkat dapat menyebabkan gradien mengeksponensial kecil saat propagasi balik, terutama dengan fungsi aktivasi sigmoid/tanh. Solusi: Gunakan ReLU dan teknik inisialisasi bobot tepat.
2. **Komputasi Memori:** Feature maps yang besar membutuhkan penyimpanan nilai aktivasi selama forward pass untuk backpropagation. Teknik checkpointing dapat digunakan untuk menghemat memori.
3. **Parameter Sharing:** Memperbarui kernel yang sama di seluruh peta fitur memerlukan akumulasi gradien dari semua lokasi spasial sebelum pembaruan bobot.
4. **Kecepatan Konvergensi:** Optimasi adaptive learning rate (Adam, RMSProp) sering diperlukan karena sensitivitas konvolusi terhadap perubahan bobot.

[!cornell] ##### Summary **Backpropagation dalam CNN** memungkinkan pelatihan end-to-end dengan menghitung gradien error terhadap kernel konvolusi dan bobot fully-connected melalui propagasi balik. Proses ini melibatkan **operasi konvolusi terbalik** untuk menghitung gradien kernel, memanfaatkan **parameter sharing** untuk efisiensi komputasi. Tantangan utama meliputi **manajemen memori** untuk nilai aktivasi dan **strategi optimasi** untuk menangani vanishing gradient. Implementasi praktis memerlukan **akumulasi gradien spasial** dan **penyesuaian hyperparameter** untuk mencapai konvergensi stabil.

[!ad-libitum]- Additional Information

Implementasi Matriks Jacobian

Untuk kernel konvolusi berdimensi tinggi, gradien $\partial H / \partial K$ dapat direpresentasikan sebagai matriks Jacobian dimana setiap elemen $J[i,j] = \partial H_i / \partial K_j$. Pada implementasi nyata, operasi ini dioptimasi menggunakan: - **im2col**: Mengubah blok input menjadi

matriks kolom - **GEMM (General Matrix Multiply)**: Perkalian matriks teroptimasi BLAS

Kompleksitas komputasi turunan: $O(n^2 * m^2)$ untuk kernel $m \times m$ dan input $n \times n$.

Algoritma Optimasi Khusus CNN

1. **AdaGrad for Convolutions**: Menyesuaikan learning rate per parameter kernel
2. **Hessian-Free Optimization**: Pendekatan kuasi-Newton untuk permukaan loss non-konveks
3. **Batch Normalization**: Menstabilkan distribusi input ke lapisan konvolusi

Perbandingan Kompleksitas

Operasi	Forward Pass	Backward Pass
Konvolusi 3x3	$O(n^2 k^2)$	$O(n^2 k^4)$
Fully-Connected	$O(mn)$	$O(mn)$
Max-Pooling	$O(n^2)$	$O(1)$

Self-Exploration Projects

1. **Implementasi Backprop Manual**: Bangun CNN mini dengan numpy (tanpa framework) yang mendukung:
 2. Konvolusi 2D dengan stride dan padding
 3. Perhitungan gradien kernel
 4. Update parameter dengan SGD
5. **Visualisasi Gradien**: Gunakan teknik guided backpropagation untuk memvisualisasikan fitur yang dipelajari kernel pada berbagai lapisan.

Tools dan Framework

- **PyTorch**: `torch.nn.gradconv2d` untuk kalkulasi gradien eksplisit
- **TensorFlow**: `tf.GradientTape` dengan operasi konvolusi khusus
- **CuDNN**: Primitive `cudnnConvolutionBackwardFilter` untuk akselerasi GPU

Further Reading

- "Deep Learning" Goodfellow et al. (Ch. 9: Convolutional Networks)
- "Neural Networks and Deep Learning" Aggarwal (Ch. 8: Backpropagation CNNs)

- Paper: "Backpropagation Applied to Handwritten Zip Code Recognition" (LeCun 1989)
- Tutorial: "Deriving Backpropagation for CNNs" (<https://arxiv.org/abs/1605.09338>)